ARQUITETURA, PESQUISA ENTRE PARES E IMPLEMENTAÇÃO

AISafe Flight Management System

Análise do estado do projeto com base no código e na configuração existentes.

UNIDADE CURRICULAR
Sistemas Distribuídos · LETI-SIDIS

ANO LETIVO
2026/2027

O AISafe divide o domínio em três serviços, apoiados por um módulo comum. Existem APIs REST, armazenamento isolado e consultas entre pares; a configuração, porém, não demonstra replicação de dados nem um cluster pronto a operar.

1. Estrutura da solução

MÓDULO	RESPONSABILIDADE PRINCIPAL	TECNOLOGIA / PAPEL
Airports & Routes	Aeroportos, pistas, certificações e rotas.	Spring Boot; cliente HTTP para aeronaves e voos.
Aircraft & Maintenance	Modelos e frota de aeronaves, manutenção e peças.	Spring Boot; encaminhamento GET entre instâncias.
Flight Operations	Agendamento e consulta de voos.	Spring Boot; integrações HTTP com os outros domínios.
Shared Core	Componentes comuns de segurança, domínio e pesquisa P2P.	Módulo Maven reutilizado por serviços dependentes.

Porquê esta divisão? Os três serviços correspondem a áreas de negócio com responsabilidades próprias: infraestrutura e rotas, frota e manutenção, e operações de voo. Separá-las reduz o acoplamento e permite desenvolver, testar e executar cada área de forma mais independente. O *Shared Core* concentra componentes técnicos reutilizados, como segurança e pesquisa P2P, sem unificar os dados dos serviços. A contrapartida é a comunicação por HTTP: operações que cruzam domínios dependem das APIs e da disponibilidade dos serviços envolvidos.

Stack identificada

Java 21, Spring Boot 4.0.4, Maven, Spring Data JPA, Hibernate e H2.

APIs REST com documentação OpenAPI; autenticação JWT e autorização por perfis.

Organização interna

Os serviços separam domínio, casos de uso e infraestrutura, incluindo adaptadores de persistência e controladores.

As integrações entre domínios são feitas por HTTP, sem transação distribuída.

2. Dados e limites da replicação

Cada serviço configura a sua própria base H2 em memória. As instâncias configuradas também usam URLs H2 distintas: isto isola os dados, mas significa que uma escrita numa instância não é automaticamente copiada para as restantes. As configurações JPA usam `create-drop`, pelo que os dados não constituem armazenamento durável após o ciclo de vida da aplicação.

DISTRIBUIÇÃO

3. Dois mecanismos de consulta entre pares

O código contém dois fluxos distintos. Ambos consultam instâncias remotas por HTTP, mas diferem no gatilho, no protocolo e na forma de limitar o encaminhamento.

A. Encaminhamento de GET — Aircraft & Maintenance

O `PeerForwardingAspect` tenta primeiro o caso de uso local. Quando este devolve uma exceção de recurso inexistente, percorre a lista estática `peers.urls` e reenvia o GET. Devolve a primeira resposta bem-sucedida; se todos os pares falharem ou não encontrarem o recurso, mantém a resposta local de não encontrado.

GET no nó inicial → **Pesquisa local** → **GET sequencial aos pares** → **Primeiro resultado ou 404**

O cabeçalho `X-Peer-Forwarded` impede novo encaminhamento circular; os tempos limite de ligação e leitura são 3 segundos. O pedido conserva o cabeçalho de autorização recebido. Este fluxo não propaga operações de escrita.

B. Pesquisa distribuída — Shared Core

O endpoint `POST /api/p2p/search` recebe um identificador de pedido, um caminho e um TTL. Cada nó pesquisa localmente e, enquanto houver TTL, encaminha a pesquisa aos pares configurados, reduzindo o contador em cada salto. Os resultados são agregados; pedidos repetidos são filtrados pelo `SearchRequestTracker`.

Pedido + requestId → **Pesquisa local** → **Pares com TTL − 1** → **Agregação JSON**

A lista deste protocolo é `p2p.peers`, no formato `peer-id=http://host:porta`; não é a mesma propriedade que `peers.urls`. A configuração predefinida deixa `p2p.peers` vazia. No código revisto, a pesquisa local concreta disponível é a de aeroportos.

4. Segurança e transporte

Autenticação

As APIs de domínio usam JWT e regras de autorização. A pesquisa P2P exige um token de par, com perfil `PEER` e tipo `P2P`.

HTTPS / TLS

O serviço Airports & Routes disponibiliza um perfil opcional `tls`, limitado a TLS 1.3 e configurado com keystore externo.

> O perfil TLS é específico desse serviço; as configurações HTTP dos restantes serviços não demonstram encriptação de ponta a ponta. Segredos e credenciais de exemplo devem ser substituídos por valores fornecidos fora do repositório antes de qualquer utilização real.

EXECUÇÃO LOCAL

5. Portas e configuração de referência

SERVIÇO	CONFIGURAÇÃO-BASE	OUTRAS INSTÂNCIAS DOCUMENTADAS
Airports & Routes	`8092`	Exemplos de execução em `8091` e `8092`.
Aircraft & Maintenance	`8081`	Perfis `instance1`, `instance2` e `instance3`: `8081–8083`.
Flight Operations	`${SERVER_PORT:8071}`	Exemplos de execução em `8071` e `8072`.

Valores lidos de `application.properties`, perfis de instância e `flight-operations-service/docs/COMMANDS.md`. São configurações de referência; não comprovam que os processos estejam ativos nem que as ligações entre todos os nós estejam corretamente alinhadas.

6. Conclusão

O projeto já separa os principais domínios em serviços e implementa duas formas de consulta distribuída, com controlo de loops/TTL e autenticação P2P. A arquitetura é uma base válida para explorar autonomia de serviços e encaminhamento entre pares. O estado atual, porém, deve ser descrito como **consulta distribuída sobre instâncias com dados isolados**, e não como replicação sincronizada ou disponibilidade garantida. A próxima prioridade técnica é alinhar a configuração de execução e validar o fluxo completo; requisitos de consistência e durabilidade devem ser tratados separadamente.